

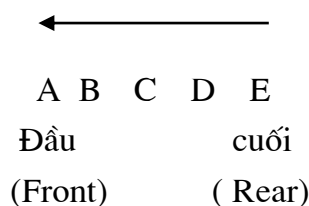
Bài 4, Tiếp Chương 2. CÁC CẤU TRÚC DỮ LIỆU CƠ BẢN

2.3. HÀNG ĐỢI - QUEUE

2.3.1. Khái niệm

Trong thực tế ta gặp nhiều trường hợp phải xử lý thông tin theo kiểu phải sắp hàng: xếp hàng chờ giải quyết công văn giấy tờ, xếp hàng vào làm thủ tục ở Cảng Hàng Không, xếp hàng mua hàng hoá tại Cửa hàng... Ai đến trước được giải quyết, phục vụ trước, ai vào sau được phục vụ sau (First in- First out. FIFO), khác với Stack là vào sau - ra trước (LIFO)

Như vậy, Hàng đợi là một danh sách dữ liệu, trong đó các phép thêm phần tử được thực hiện ở cuối danh sách (Rear), còn các phép xử lý như hiện dữ liệu, xóa phần tử được thực hiện ở đầu danh sách (điểm Front)



Trong máy tính, các nhiệm vụ tính toán, in ấn được xếp hàng chờ đợi xử lý do hệ điều hành quản lý là một ví dụ về hàng đợi được áp dụng trong tiến trình xử lý của máy.

Các phép toán cơ bản khi xử lý hàng đợi là :

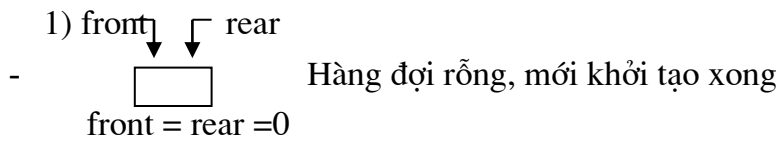
- Khai báo cấu trúc và cài đặt hàng đợi
- Khởi tạo hàng đợi ;
- Thêm phần tử vào cuối hàng đợi ;
- Cho (hiện dữ liệu) phần tử đầu hàng đợi.
- Xóa phần tử khỏi đầu hàng đợi ;
- Xác định hàng đợi có rỗng không;
-

Hàng đợi khá giống ngăn xếp, chúng ta có thể dùng mảng để cài đặt hàng đợi.

Đặt $Q[]$ – mảng hàng đợi, ngoài mảng $Q[n]$ cần dùng thêm 2 biến **front**(đầu hàng) và **rear**(cuối hàng).

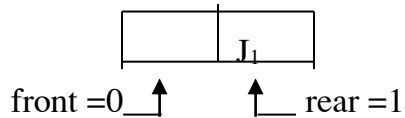
front luôn chỉ vào phần tử đầu của hàng đợi mà có thể lấy ra được, còn **rear** luôn chỉ vị trí cuối hàng đợi, nơi có thể thêm vào phần tử.

Ta có thể mô tả :



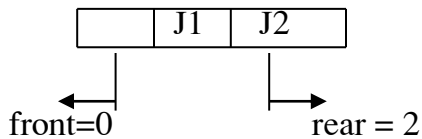
2) Đưa công việc J₁ vào hàng đợi

Q[1]



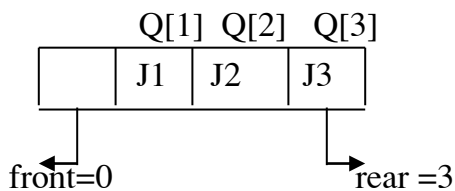
3) Công việc J₂ vào hàng đợi

Q[1] Q[2]

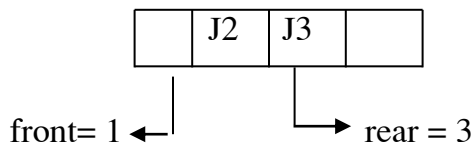


4) Thêm công việc J₃ vào hàng đợi

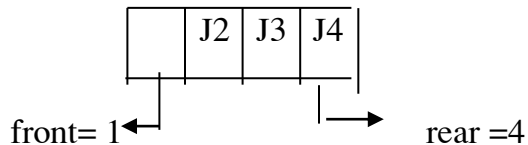
Q[1] Q[2] Q[3]



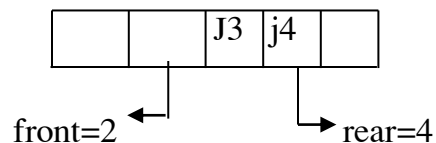
5) Đưa công việc J₁ ra khỏi hàng đợi



6) Thêm công việc J₄ vào hàng đợi



7) Đưa công việc J₂ ra khỏi hàng đợi



Trong cấu trúc này, mỗi lần lấy một công việc ra khỏi hàng đợi là phải tăng front++, thêm vào hàng đợi phải tăng rear++. Ta có thể mô tả lại cách biểu diễn trên như sau :

● ●	Q[1]	[2]	[3]	[4]	[5]	[6]	Giải thích
front	rear						
0	0	Hàng	đội	rỗng			Khởi tạo Q
0	1	j1					j1 -> Q
0	2	j1	j2				j2 -> Q
0	3	j1	j2	j3			j3->Q
1	3		j2	j3			j 1 ra khỏi Q
1	4		j2	j3	j4		j4 -> Q
2	4			j3	j4		j2 ra khỏi Q

2.3.2.Cài đặt hàng đợi bằng mảng

a) Cài đặt mảng cho hàng đợi

Với sơ đồ nêu ở trên ta có thể cài đặt mảng cho hàng đợi nh- sau.

Các biến cần sử dụng :

Q[] – mảng hàng đợi

QSize : kích thước tối đa của hàng đợi

front(*đầu hàng*) và rear(*cuối hàng*).

item – phần tử cần thêm vào Q[]

1)Thủ tục Khởi tạo hàng đợi

```
void KhoitaoQ(int Q[], int front, int rear)
{   front = 0 ;   rear = 0  ;
```

2)Kiểm tra hàng đội có rỗng không

```
bool IsEmpty(int front, int rear)
{ return (front == rear); }
```

3)Cho phần tử ở đầu hàng đợi

```
int FRONT(int Q[], int front)
{ return Q[front]; }
```

4)Thủ tục thêm phần tử vào hàng đợi:

```
void ThemQ(int Q[], int item, int &rear, int QSize)
{ if(rear == QSize)      // Hàng đợi đã đầy
  cout<< " Hang doi day \n";
  else
  { Q[rear] = item;  // Thêm phần tử vào cuối hàng đợi
    rear++;  }
}
```

5) Xóa phần tử ra khỏi đầu hàng đợi :

```
void XoaQ(int Q[], int &front, int rear)
{   if(front == rear)        // Hàng đợi rỗng
    cout<< " Hang doi trong rong - \n" ;
    else
        {Q[front] = 0;      // Xóa phần tử ở front
         front++;
        }
}
```

Để hình dung phép XoaQ, hãy xem :

Trước khi xóa :

0	40	99	56	94
0	1	2	3	4
front = 0				rear=4

Sau khi xóa 40:

	0	99	56	94
0	1	2	3	4
	front = 1			rear=4

6) Hàm cho kích thước của Hàng đợi,

```
int Size(int Q[], int front, int rear)
{   return (rear - front); }
```

Hàm này cho số lượng phần tử đang có trên hàng đợi

2.3.3. Thư viện chuẩn hàng đợi trong C++

a) Khai báo #include <queue>

Viết dòng này ngay sau dòng #include <iostream> trong phần đầu chương trình.

b) Khai báo biến hàng đợi q

queue <kiểu dữ liệu> <tên biến> ;

Ví dụ : queue<int> q;

Trong phần chương trình, sử dụng các hàm của thư viện chuẩn queue như sau.

- 1) Thủ tục đưa 1 phần tử x vào queue : q.push(x);
- 2) Lấy 1 phần tử đầu của queue (không xóa phần tử này) : x = q.front();
- 3) Xóa phần tử đầu (không trả về giá trị): q.pop();
- 4) Kiểm tra rỗng: trả về true hoặc false : q.empty();

Ứng dụng . sử dụng thư viện chuẩn stack và queue trong C++, viết chương trình nhập dãy số rồi đưa ra dãy số đã đảo ngược :

(Chương trình sq1 trong thư mục E:\namhoc19_20\sq1)

```
#include <iostream>
#include <stack>
#include <queue>
#include <iomanip> //Điều khiển cout<<
using namespace std;
int main()
{
    queue<int> q; stack<int> s;
    int x;
    cout<< " \n    Nhập vào các số nguyên : \n ";
    char nhaptiep='Y';
    while (toupper(nhaptiep)=='Y')
    {
        cout<< " \n    Nhập số : "; cin>>x;
        q.push(x);
        cout<<" \n    Có nhập tiếp không (y/n)?"; cin>>nhaptiep;
    }
    cout<<" \n    Kết quả chuyển vào stack : \n\n"; cout<<setw(20);
    while (not q.empty())
    {
        x = q.front(); q.pop();
        s.push(x); x=s.top();
        cout<<" "; s.x= "<<x;
    }
    cout<<"\n\n"; cout<<"    Kết quả chuyển vào queue : \n\n";
    cout<<setw(20);
    while (not s.empty())
    {
        x=s.top(); s.pop();
        q.push(x); cout<<" "; q.x= "<<x;
    }
    cout<<"\n\n"; system("pause");
    return 0;
}
```

Kết quả chạy chương trình :

Nhap vao cac so nguyen :

```
Nhap so : 1
Co nhap tiep khong (y/n)?y
Nhap so : 2
Co nhap tiep khong (y/n)?y
Nhap so : 3
Co nhap tiep khong (y/n)?y
Nhap so : 4
Co nhap tiep khong (y/n)?n
```

Ket qua nhap vao stack : ; S.x= 1; S.x= 2; S.x= 3; S.x= 4

Ket qua chuyen vao queue : ; Q.x= 4; Q.x= 3; Q.x= 2; Q.x= 1

2.3.4.Lớp hàng đợi

Chương trình D:\2021.08.21 Hoang Nghia Ty Drive\Caclopday\namhoc21_22\classQueue\

```
# include <iostream>
using namespace std;
# define Max 15
class queue
{private:
    int a[Max];
    int front; int rear;
public:
    queue();
    void insert(int i);Thêm vào đầu hàng đợi
    void HienRear()
    {rear--;
        cout<<" rear ="<<this->rear<<" , q.a["<<this->rear<<"]="<<this->a[rear]<<"\n";
        rear++;
        return;
    };
    int remove();// Xóa
    void HienFront()
    {cout<<" front ="<<this->front<<" , q.a["<<this->front<<"]="<<this->a[front]<<"\n";
        return;
    };
    int isEmpty();
    int isFull();
};// Hết định nghĩa lớp class queue
queue::queue() //khởi tạo hàng đợi rỗng
{ front=0; rear=0; }

void queue::insert(int i) //Thêm phần tử vào cuối hàng đợi
{ if(isFull()) { cout<<"\n Hang doi DAY — \n"; return; }
    a[rear] = i;
    rear++;
} // Hết khai báo hàm insert(int i)
```

```

int queue::remove() //Xóa phần tử ở đầu hàng đợi
{ if(isEmpty())
    { cout<<"\n  Hang doi RONG- \n"; return (- 1); }
    return(a[front++]);
} //*****Hết khai báo hàm remove()
int queue::isEmpty()
{ if(front == rear)      return 1;
    else return 0;
} //***** Hết khai báo hàm isEmpty()
int queue::isFull()
{ if(rear == Max)      return 1;
    else return 0;
} //***** Hết khai báo hàm isFull()

int main()
{ system("cls");// Xóa màn hình
queue q; //khai báo biến hàng đợi q
q.insert(1);//Thêm 1
q.HienRear();system("pause");//Hiện cuối hàng đợi
q.insert(3);//Thêm 3
q.HienRear();system("pause");//Hiện cuối hàng đợi
q.insert(5);//Thêm 5
q.HienRear();system("pause");//Hiện cuối hàng đợi
q.remove();//xóa 1
q.HienFront();system("pause");//Hiện đầu hàng đợi
q.remove();//Xóa 3
q.HienFront();system("pause");//Hiện đầu hàng đợi
q.insert(7);//Thêm 7
q.HienRear();system("pause");//Hiện cuối hàng đợi
q.remove();//Xóa 5
q.HienFront();system("pause");//Hiện đầu hàng đợi
q.insert(9);//Thêm 9
q.HienRear();//Hiện cuối hàng đợi
system("pause");
return 0; }
}

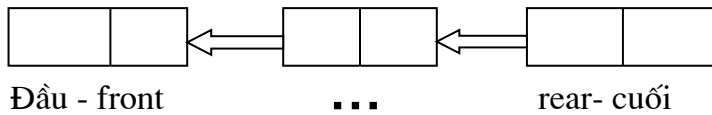
```


2.3.5. Hàng đợi liên kết

2.3.5.1. Khái niệm

Trong hàng đợi liên kết, các phần tử được nối kết với nhau bằng con trỏ liên kết. Sẽ sử dụng 2 con trỏ **front** và **rear** để trỏ vào đầu và cuối hàng đợi liên kết.

Các thao tác thêm phần tử vào hàng đợi liên kết sẽ thực hiện ở cuối hàng đợi, thao tác xử lý hay xóa phần tử hàng đợi liên kết sẽ thực hiện ở đầu hàng đợi:



2.3.5.2. Cài đặt hàng đợi liên kết bằng danh sách liên kết

Trong mục danh sách liên kết đã xét các thao tác khởi tạo danh sách, thêm, hiện, xóa các phần tử của danh sách. Hàng đợi liên kết là trường hợp riêng của danh sách liên kết, khi mà thao tác xử lý dữ liệu hay xóa một phần tử chỉ thực hiện ở đầu danh sách, còn thêm phần tử chỉ thực hiện ở cuối danh sách.

Các bước cài đặt hàng đợi liên kết bằng danh sách liên kết :

1. Cấu trúc hàng đợi liên kết

a) Cấu trúc nút-phần tử hàng đợi liên kết

```
struct Node
{
    int du_lieu;
    Node *next;
};
```

b) Tạo một cấu trúc của hàng đợi liên kết

Sử dụng Node để tạo hai con trỏ là **front** và **rear** để quản lý phần tử đầu và phần tử cuối.

```
struct Queue
{
    Node *front, *rear;
};
```

c) Khởi tạo hàng đợi liên kết rỗng ban đầu

```
void KhoitaoQ(Queue &q)
{q.front = q.rear = NULL; }
```

d) Tạo nút mới cho hàng đợi liên kết

```
Node *KhoitaoNode( int x )
{
    Node *p = new Node; // Khai báo con trỏ p có kiểu Node, được cấp phát bộ nhớ theo kiểu Node
    p->next = NULL; // con trỏ p chưa trỏ vào đâu cả
    p->du_lieu = x; // dữ liệu của p được gán = x
    return p; // kết quả trả về phần tử p mới tạo để dùng vào việc đưa vào hàng đợi
}
```

2. Hàm Push –Nhập thêm phần tử vào cuối hàng đợi liên kết

Bước 1. kiểm tra xem hàng đợi có rỗng hay không, nếu rỗng thì phần tử được thêm vào chính là p.front và p.rear, nếu không rỗng thì xuống bước 2.

Bước 2. Cho p.next của p.rear trở tới node p

```
void Push(Queue &q, Node *p ){ // thêm phần tử vào cuối hàng đợi
    if(!q.front) q.front = q.rear = p; //nếu q rỗng thì q.front và q.rear được gán = p
    else //nếu không thì
        { q.rear->next = p; // cho next của q.rear trở tới node p
          q.rear = p; //khi đó node p chính là q.rear
        }
} //câu hỏi : vì sao !q.front ĐÚNG thì q rỗng ?
```

3.Hàm Top- lấy(hiện) phần tử đầu hàng đợi Queue liên kết

Bước 1. kiểm tra nếu hàng đợi rỗng thì return 0 và kết thúc hàm, nếu không thì xuống bước 2.

Bước 2. return phần tử đầu hàng đợi liên kết.

```
int Top(Queue q ){
    if(q.front){ //kiểm tra nếu hàng đợi tồn tại phần tử
        return q.front->du_lieu; // thì ta thực hiện return phần tử đầu tiên
    }
    else return 0; //Nếu hàng đợi rỗng thì return 0 và kết thúc hàm
}
```

4.Hàm Pop- xóa phần tử đầu hàng đợi liên kết

```
int Pop(Queue &q ) // hàm xóa phần tử đầu trong hàng đợi
{ if(q.front) //kiểm tra nếu hàng đợi tồn tại thì thực hiện các câu lệnh
{ Node *p = q.front; // khai báo *p để lưu nút đầu hàng đợi
  q.front = p->next; // đầu hàng đợi trở đến nút tiếp theo của chính nó
  int a = p->du_lieu; // trả về du_lieu của nút ở đầu hàng đợi sẽ xóa
  delete p; } // xóa nút đầu hàng đợi
return 0; // nếu hàng đợi rỗng
}
```

5.Hiện các phần tử của hàng đợi

```
void Hien(Queue q ) // Hiện các phần tử trong hàng đợi
{ Node *p = q.front;
  while(p){ cout<<p->du_lieu <<setw(5);
            p = p->next; }
  cout<< endl;
}
```

6.Xóa cả hàng đợi

```
void XoaQ(Queue q) // Xóa các phần tử trong hàng đợi
{
    while(q.front != NULL)
    {
        Node *p = q.front;
        q.front = p->next;
        delete p;
    }
    q.rear = NULL;
}
```

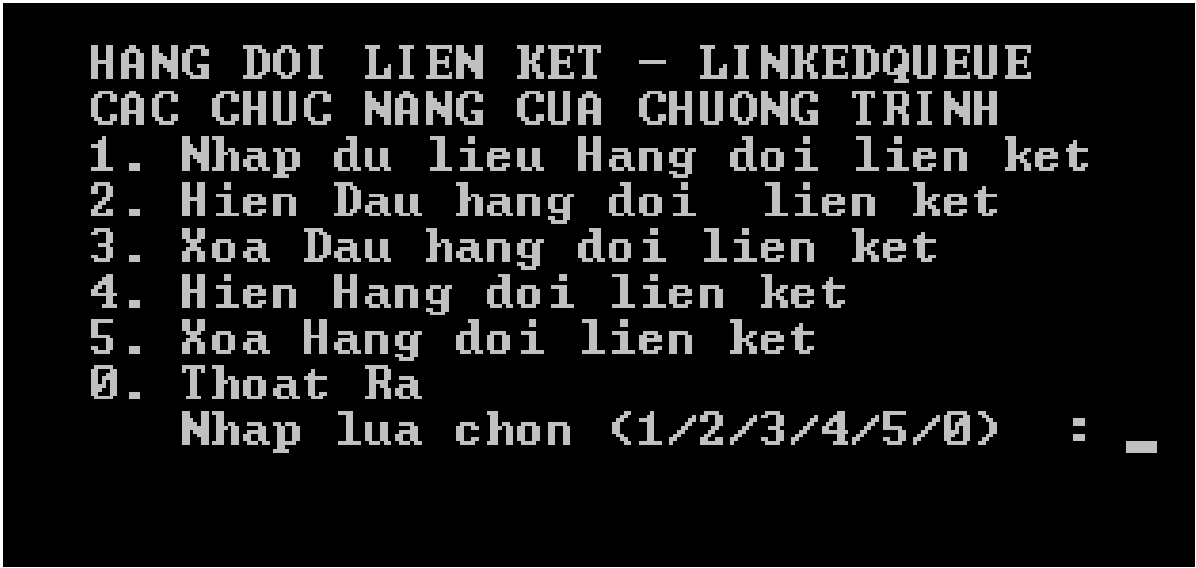
Bài tập : ghép nối các khai báo, các hàm đã học thành một chương trình hoàn chỉnh

Có 5 chức năng :

1. Nhập dữ liệu vào hàng đợi,
2. Hiện đầu hàng đợi
3. Xóa đầu hàng đợi
4. Hiện hàng đợi liên kết
5. Xóa hàng đợi liên kết
0. Thoát ra

Gợi ý : xem màn hình đã chạy thử chương trình để tổ chức chương trình .

Kết quả chạy chương trình : hiện ra menu có các chức năng :



```
HANG DOI LIEN KET - LINKEDQUEUE
CAC CHUC NANG CUA CHUONG TRINH
1. Nhập dữ liệu Hàng đợi liên kết
2. Hiện Đầu hàng đợi liên kết
3. Xóa Đầu hàng đợi liên kết
4. Hiện Hàng đợi liên kết
5. Xóa Hàng đợi liên kết
0. Thoát Ra
Nhập lựa chọn (1/2/3/4/5/0) : _
```

Kết quả lựa chọn thực hiện các chức năng :

Nhap phan tu thu 5 cua Hang Doi: 5

HANG DOI LIEN KET - LINKEDQUEUE

CAC CHUC NANG CUA CHUONG TRINH

1. Nhap du lieu Hang doi lien ket
2. Hien Dau hang doi lien ket
3. Xoa Dau hang doi lien ket
4. Hien Hang doi lien ket
5. Xoa Hang doi lien ket
0. Thoat Ra

Nhap lua chon <1/2/3/4/5/0> : 2

Hien Phan tu Front cua Hang Doi : 1

HANG DOI LIEN KET - LINKEDQUEUE

CAC CHUC NANG CUA CHUONG TRINH

1. Nhap du lieu Hang doi lien ket
2. Hien Dau hang doi lien ket
3. Xoa Dau hang doi lien ket
4. Hien Hang doi lien ket
5. Xoa Hang doi lien ket
0. Thoat Ra

Nhap lua chon <1/2/3/4/5/0> : 3

Da xoa - Delete phan tu o Front cua Hang Doi

HANG DOI LIEN KET - LINKEDQUEUE

CAC CHUC NANG CUA CHUONG TRINH

1. Nhap du lieu Hang doi lien ket
2. Hien Dau hang doi lien ket
3. Xoa Dau hang doi lien ket
4. Hien Hang doi lien ket
5. Xoa Hang doi lien ket
0. Thoat Ra

Nhap lua chon <1/2/3/4/5/0> : 4

Hien cac phan tu trong Hang Doi Queue :

2 3 4 5

HANG DOI LIEN KET - LINKEDQUEUE

CAC CHUC NANG CUA CHUONG TRINH

1. Nhap du lieu Hang doi lien ket
2. Hien Dau hang doi lien ket
3. Xoa Dau hang doi lien ket
4. Hien Hang doi lien ket
5. Xoa Hang doi lien ket
0. Thoat Ra

Nhap lua chon <1/2/3/4/5/0> :